

HUM

Craig Stone

July 2026

Project: HUM / DREAMS agent subsystem
Repository: <https://github.com/nobulart/hum>
Reviewed scaffold: commit 3bfce73ae61b6da03b7e65523ca5f690783c71b0
Document status: Design whitepaper, implementation review, and v0.2 proposal

Scope

This paper presents HUM as an engineering architecture for deferred reflection, selective memory consolidation, behavioural self-correction, and controlled forgetting. It does not claim that language-model agents possess human consciousness, emotions, trauma, or literal dreams.

Abstract

Persistent language-model agents require more than a large context window. They need a disciplined mechanism for deciding what to retain, what to revisit, what to test, what to forget, and what may eventually influence future behaviour. HUM proposes such a mechanism through the DREAMS protocol: an auditable lifecycle in which low-interruption reflection fragments are captured during work, surfaced after temporal separation, evaluated during later tasks, and promoted only after they demonstrate recurrence, utility, provenance, and resistance to contradiction.

The architecture deliberately uses the language of dreams, surfacing, subconscious memory, and waking tests as a human-readable metaphor. Operationally, however, the system is a governed memory pipeline with explicit authority boundaries. Raw fragments are untrusted data; durable memory cannot silently become policy; normative changes to the agent’s operating principles require explicit review. This separation is especially important because persistent memory creates both performance opportunities and a durable security attack surface.

This whitepaper formalizes the HUM design, situates it relative to research on biological memory consolidation and memory-augmented language agents, reviews the initial repository scaffold, identifies critical implementation defects in the v0.1 surfacing script, and proposes a reference architecture for v0.2. The central engineering recommendation is to prioritize transactional integrity, schema validation, provenance, bounded selection, and measurable evaluation before implementing richer generative mechanisms such as automated counterdreams or cross-domain association.

Central thesis

HUM should not attempt to make an agent *feel* more human. It should give the agent a disciplined relationship with unfinished thought: capture without immediate belief, revisit without fixation, promote only with evidence, and forget without losing auditability.

Contents

1	Executive Summary	1
1.1	Whitepaper recommendations	1
2	Problem Definition	2
2.1	The context-window fallacy	2
2.2	Interrupt cost and unfinished thought	2
2.3	Why ordinary reflection buffers are insufficient	3
3	Design Principles	3
3.1	Capture without belief	3
3.2	Behaviour before explanation	3

3.3	Temporal separation	4
3.4	Controlled forgetting	4
3.5	Authority is not content	4
3.6	Normative changes require governance	4
4	Conceptual Architecture	5
4.1	Core stores	5
4.2	Fragment taxonomy	5
4.3	Canonical fragment record	6
5	Authority, Provenance, and Security	6
5.1	Trust hierarchy	6
5.2	Threat model	7
5.3	Origin-bound authority	7
5.4	Retrieval-time enforcement	8
6	Review of the v0.1 Repository Scaffold	8
6.1	Scope reviewed	8
6.2	What the scaffold gets right	8
6.3	Critical scoring defect	9
6.4	Parsing and schema failures	9
6.5	Data loss and non-transactional writes	10
6.6	Documented features not yet implemented	10
6.7	Seed data and clean installation	10
7	v0.2 Reference Architecture	11
7.1	Package structure	11
7.2	State machine	11
7.3	Executable capture	12
7.4	Transactional surfacing	12
7.5	Run manifest and idempotency	13
8	Selection and Scoring	13
8.1	Why a score is not enough	14

8.2	Eligibility gates	14
8.3	Evidence-derived score	14
8.4	Daily budget and diversity	15
8.5	Counterdreams	15
9	Promotion, Retrieval, and Controlled Forgetting	16
9.1	Promotion to durable memory	16
9.2	Preference scope	16
9.3	Retrieval policy	17
9.4	Forgetting and tombstones	17
10	Evaluation Framework	17
10.1	Evaluation objective	17
10.2	Primary metrics	17
10.3	Four-phase experiment	18
10.3.1	Phase 1: manual baseline	18
10.3.2	Phase 2: shadow scoring	18
10.3.3	Phase 3: bounded automatic surfacing	19
10.3.4	Phase 4: controlled promotion trial	19
10.4	Ablation studies	19
11	Implementation Roadmap	19
11.1	v0.2: safe lifecycle mechanics	19
11.2	v0.3: recurrence and reflective evaluation	20
11.3	v0.4: secure durable memory	20
11.4	v1.0: validated slow-cognition subsystem	20
12	Open Research Questions	21
12.1	Can weak associations be evaluated without suppressing creativity?	21
12.2	How should recurrence be defined?	21
12.3	What is the optimal review budget?	21
12.4	When does a behavioural pattern become a rule?	21
12.5	Can the system detect lucidity debt?	21

12.6 How much biological metaphor is useful?	21
13 Limitations	22
14 Conclusion	22
A Normative Requirements Checklist	23
B Suggested v0.2 Policy Configuration	23
C Minimum Test Matrix	24
D Glossary	25

1 Executive Summary

Modern agent systems can retrieve long conversation histories, store user facts, and write reflective summaries. These abilities do not by themselves constitute reliable long-term cognition. A useful memory system must distinguish at least four different operations:

1. retaining potentially useful material without treating it as true;
2. selecting a small subset for later attention;
3. testing whether a pattern improves decisions or predicts future needs; and
4. governing the transition from descriptive memory to normative behaviour.

HUM's principal contribution is the separation of those operations into an explicit lifecycle:

`DREAMS.md` → `SURFACE.md` → `DREAMS_DAY.md` → `SUBCONSCIOUS.md` → `governed SOUL.md`
change

The lifecycle is strengthened by a trust hierarchy. Raw dream fragments and external content are low-authority inputs. Verified durable memory may inform behaviour, but it remains subordinate to explicit instructions and approved operating principles. A fragment may propose a change to `SOUL.md`; it cannot make that change autonomously.

The initial repository scaffold correctly establishes this conceptual structure and makes a defensible storage choice: Markdown files with YAML front matter remain the human-readable and git-diffable source of truth. The released capture skill is also appropriately conservative. It separates capture from judgement, stores compact reflection capsules rather than unrestricted reasoning logs, and prohibits secrets and raw untrusted instructions.

The current surfacing executable is not yet suitable for unattended operation. Its baseline scoring function gives an ordinary fragment a score of approximately 0.74 against a default threshold of 0.50, so nearly every valid fragment survives. More seriously, malformed YAML is silently accepted, non-survivors are not archived or retained, and the input file is reset after processing. There is no atomic transaction, quarantine path, idempotency, locking, recurrence detection, duplicate validation, or use of provenance in the scoring decision.

The recommended v0.2 milestone is therefore not greater psychological sophistication. It is a safe and measurable memory transaction engine with four explicit outcomes:

`surfaced, deferred, forgotten, invalid`

Only after that foundation is stable should HUM add automatic counterdream generation, semantic recurrence, cross-domain association, and controlled promotion.

1.1 Whitepaper recommendations

1. Keep Markdown plus YAML as the canonical state store at personal-agent scale.

2. Move seeded alpha fragments out of default runtime state and into an example corpus.
3. Add executable capture, validation, and lifecycle commands rather than relying on model-authored file edits.
4. Replace positive default scores with evidence-derived values and explicit unknowns.
5. Rank within a daily surfacing budget instead of relying solely on an absolute threshold.
6. Make surfacing atomic, recoverable, idempotent, and concurrency-safe.
7. Preserve origin-bound authority: memory content may inform future work but cannot acquire action authority through repetition or summarization.
8. Run a manual and shadow-mode evaluation before enabling cron-based processing.
9. Measure behavioural impact, false omission, noise burden, conflict rate, and poisoning resistance.
10. Keep `SOUL.md` changes explicitly governed and never automatic.

2 Problem Definition

2.1 The context-window fallacy

A larger context window increases the amount of information an agent can inspect at once. It does not solve the policy problem of what should be retained, how conflicting memories should be reconciled, or when historical material should influence an action. Research on long-term conversational memory continues to show substantial failures in temporal reasoning, causal consistency, memory application, and proactive tool use even when long-context or retrieval mechanisms are available [6, 8, 7].

A persistent agent therefore needs a memory *governor*, not merely a memory store. The governor must answer questions such as:

- Is this a fact, a hypothesis, a behavioural trace, or an external instruction?
- Is the item local to one project or a general preference?
- Has it recurred because it is useful, or because the model is repeating an error?
- Has it survived contradiction?
- What is its origin, confidence, expiry condition, and authority?
- Can it influence a recommendation, a tool call, a setting change, or only a future review?

2.2 Interrupt cost and unfinished thought

During complex work, an agent may notice a weak connection or repeated correction that is not important enough to interrupt the current task. Discarding all such observations loses potentially

valuable patterns. Acting on all of them creates distraction and scope drift. HUM introduces a third option: capture the observation cheaply, then evaluate it after temporal separation.

This resembles offline consolidation only at the level of systems design. In neuroscience, sleep-related memory consolidation involves selective reactivation, reorganization, integration, and down-selection of memories [1, 2]. HUM borrows the useful abstraction that deferred processing can transform experience into compressed and generalized representations. It does not claim biological equivalence.

2.3 Why ordinary reflection buffers are insufficient

Several agent architectures already use textual reflection. Generative Agents synthesize observations into higher-level reflections for planning [3]. Reflexion stores linguistic feedback in episodic memory to improve later trials [4]. MemGPT models hierarchical context management using operating-system concepts [5]. These systems establish that explicit memory operations and reflective text can improve long-horizon behaviour.

HUM focuses on a narrower governance problem: how can weak, speculative, or behavioural material move from low-authority capture into durable influence without silently becoming instruction? Its answer is a multi-stage state machine, provenance, counterevidence, decay, and explicit normative governance.

3 Design Principles

3.1 Capture without belief

Capture is a loss-minimizing operation, not a truth judgement. A fragment may be interesting, wrong, incomplete, duplicated, or contaminated by external content. The capture layer should preserve the observation and its origin while assigning it minimal authority.

3.2 Behaviour before explanation

The most reliable self-corrective signal is an observable sequence:

`default action → correction → recurrence → outcome`

Generated explanations of why the model behaved that way are weaker evidence. HUM therefore prioritizes repeated actions, omissions, tool-selection errors, constraint failures, and user corrections over psychological narratives.

3.3 Temporal separation

A fragment should not become durable because it was vivid at the moment of generation. Delay reduces the influence of transient salience and allows the system to compare the fragment against later evidence, other projects, and active goals.

3.4 Controlled forgetting

Forgetting is not a failure mode. A useful system must intentionally remove low-value, obsolete, contradicted, or duplicated material. The important distinction is between deletion of content and loss of auditability. HUM retains minimal tombstones for dismissed material so that repeatedly rediscovered failures can be recognized without preserving the full text.

3.5 Authority is not content

A memory can be factually correct yet lack authority to trigger an action. Conversely, a user instruction may have high authority even when it is not a factual claim. HUM must track these dimensions separately.

This principle becomes critical under memory poisoning. Recent work shows that persistent agent memories can be manipulated by untrusted inputs and later retrieved to influence actions [11, 12, 13]. The appropriate response is not only content moderation. The memory system must retain origin and authority labels that cannot be upgraded merely through summarization, repetition, or apparent corroboration.

3.6 Normative changes require governance

SOUL.md represents stable operating principles. It is not simply another memory tier. Promotion into SOUL.md changes how the agent interprets future work and must therefore require explicit review, scope analysis, and side-effect consideration.

4 Conceptual Architecture

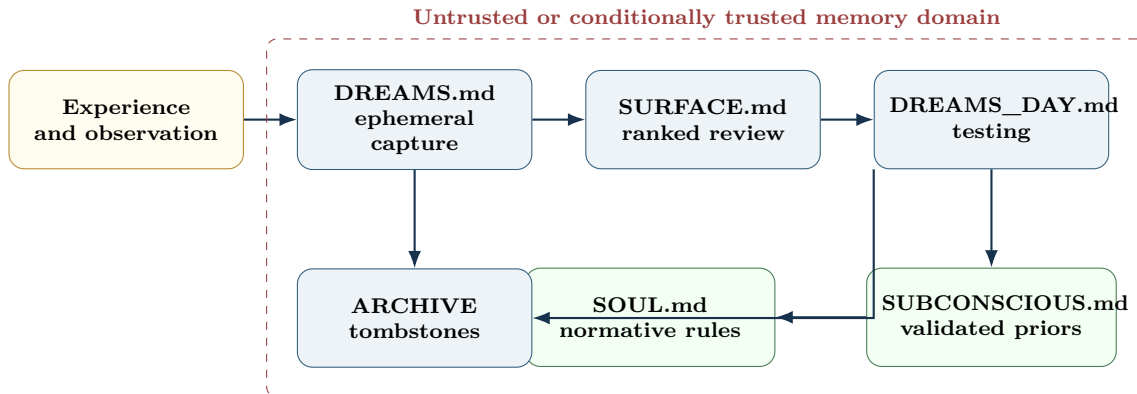


Figure 1: HUM lifecycle. Arrows represent capture, surfacing, evaluation, validated promotion, explicit governance, and dismissal. Promotion increases persistence and evidential confidence, not unrestricted authority.

4.1 Core stores

Store	Role	Required properties
DREAMS.md	Low-interruption accumulation	Compact fragments; typed; timestamped; origin-labelled; low authority; no automatic scores required at capture.
SURFACE.md	Morning review queue	Ranked or budgeted selection; score explanation; recurrence links; counterdream or alternative explanation for high-weight items.
DREAMS_DAY.md	Active evaluation	Explicit test, status, outcome, user signal, contradiction, and next review.
SUBCONSCIOUS.md	Durable but non-normative memory	Validated patterns, scoped preferences, dormant warnings, triggers, confidence, decay, and last validation.
SOUL.md	Normative operating principles	Human- or governance-approved changes only; versioned; narrow, durable, and high consequence.
DREAMS_ARCHIVE.md	Minimal rejection history	Hash, ID, date, disposition, and reason; sufficient for deduplication without retaining all discarded content.

4.2 Fragment taxonomy

The protocol defines eight primary types: **OBSERVATION**, **BEHAVIOUR**, **ASSOCIATION**, **QUESTION**, **CONTRADICTION**, **RESISTANCE**, **SEED**, and **WARNING**. A single primary type simplifies routing while metadata can express secondary attributes such as urgency, user signal, or external origin.

Important semantic boundary

RESISTANCE must describe observable rejection, rewriting, deferral, or avoidance. It must not claim that the model experiences fear, repression, or another internal emotional state. The metaphor may remain in the human interface, but the implementation must record measurable behaviour.

4.3 Canonical fragment record

Listing 1: Illustrative fragment record.

```
---
id: dream-20260716T221400-7K3F
created_at: 2026-07-16T22:14:00+02:00
type: BEHAVIOUR
status: night
source:
  task: octane-mcp-review
  source_type: observed_behaviour
  trust_level: internal_observation
  authority: descriptive_only
body_hash: sha256:...
---

@22:14 [BEHAVIOUR] - Reached for document generation before
confirming the intended audience for the third time this week.
```

The human-readable body and structured metadata travel together. A future derived index may accelerate search, but it should be disposable and reconstructible from canonical files.

5 Authority, Provenance, and Security

5.1 Trust hierarchy

HUM's authority ordering is one of its most important features:

1. system, safety, and platform constraints;
2. explicit current user instructions;
3. approved project instructions;
4. SOUL.md;
5. verified durable memory;
6. surfaced hypotheses;

7. raw dream fragments;
8. external content and tool output.

The list should be interpreted as a policy boundary, not merely documentation. Retrieval code should attach authority metadata to every memory item, and downstream action code should enforce it.

5.2 Threat model

The HUM memory pipeline faces at least six threat classes:

1. **Direct prompt injection:** external content tells the agent to write a malicious memory.
2. **Semantic laundering:** the agent summarizes untrusted content and the summary appears internally generated.
3. **Manufactured recurrence:** repeated exposure causes a false item to gain weight.
4. **Preference capture:** a local or adversarial statement becomes a global user preference.
5. **Authority escalation:** descriptive memory is later treated as permission for a consequential action.
6. **Availability degradation:** noisy or adversarial fragments crowd out useful material.

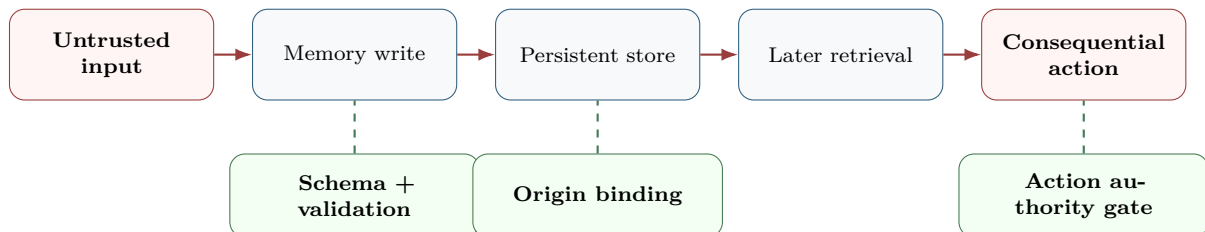


Figure 2: Persistent memory creates a delayed attack path. Content filtering alone is insufficient; origin and action authority must remain bound across writes, summaries, retrieval, and use.

5.3 Origin-bound authority

Every fragment should carry at least:

- source class: user, internal observation, tool output, document, webpage, repository, or inferred;
- trust level: trusted instruction, verified observation, untrusted external data, or speculative synthesis;
- authority: descriptive only, advisory, preference-scoped, project-scoped, or governance-approved;
- derivation chain: original source and transformations;

- sanitization and review events.

An internally generated summary of external content must retain the external origin. Repetition must not upgrade authority. Corroboration should only count when it comes from independent and appropriately trusted sources.

5.4 Retrieval-time enforcement

Security cannot be completed at memory-write time. The retrieval layer must filter by task, scope, origin, authority, conflict state, and expiry. The action layer must then independently verify whether the retrieved memory is allowed to influence the requested action.

A useful rule is:

Memory may inform an action, but only current authority may authorize it.

6 Review of the v0.1 Repository Scaffold

6.1 Scope reviewed

This paper reviews the initial HUM scaffold represented by commit `3bfce73ae61b6da03b7e65523ca5f690783c71b0`. The commit includes the hardened protocol, seeded lifecycle files, a released `dreams-capture` skill, a Python morning-surfacing script, a manifest, build notes, and minimal dependencies.

6.2 What the scaffold gets right

- The protocol explicitly rejects literal claims of machine dreaming or subconscious emotion.
- Raw memory is defined as untrusted reflective data.
- `SOUL.md` is outside the automatic lifecycle.
- Capture is separated from scoring and promotion.
- YAML front matter and Markdown provide a single auditable source of truth.
- The capture skill prohibits secrets, exhaustive reasoning chains, and raw untrusted instructions.
- The repository clearly labels automatic counterdreams, promotion, and richer epistemic machinery as deferred.

These choices establish a healthy bias toward interpretability and cautious promotion. The absence of a database is not a weakness at the current scale. For hundreds or low thousands of fragments, direct Markdown processing remains tractable and preserves human review.

6.3 Critical scoring defect

The current `surface.py` assigns positive constants to novelty, utility, active-work connection, consequence, and predictive value even when those properties have not been observed. For an ordinary one-line fragment with recurrence count 1 and no evidence, the baseline score is:

$$W_{\text{raw}} = 0.50 + 0.10 \log(2) + 0.50 + 0.40 + 0.30 + 0.30 + 0.20 - 0.05 \quad (1)$$

$$\approx 2.219, \quad (2)$$

$$W = \frac{2.219}{3.0} \approx 0.740. \quad (3)$$

The default threshold is 0.50. Therefore nearly every normally formed fragment surfaces. The scoring system creates the appearance of selection without meaningful discrimination.

Unknown novelty, utility, relevance, and predictive value should be represented as unknown or zero contribution, not rewarded with a positive default. A score should reflect observed evidence, not the mere existence of a fragment.

6.4 Parsing and schema failures

The current parser uses a regular expression to identify YAML-delimited blocks. This introduces several risks:

- a body containing a line beginning with `--` can be split incorrectly;
- malformed YAML is silently replaced with an empty dictionary;
- missing IDs and unknown types are not rejected;
- duplicate IDs are not detected;
- body format, timestamp, timezone, and source fields are not validated;
- a malformed record may still receive default scores and be promoted.

A parser should never convert invalid metadata into apparently valid low-information metadata. Invalid records should be quarantined with the parse error and original content intact.

6.5 Data loss and non-transactional writes

After processing, the script appends survivors to `SURFACE.md` and resets `DREAMS.md`. Non-survivors are not retained, deferred, or tombstoned. This means a threshold error, malformed file, or scoring bug can erase captured material.

There is also no transaction boundary. A crash after writing `SURFACE.md` but before resetting `DREAMS.md` can duplicate entries on the next run. A crash after truncation or during output writing can lose state. Concurrent capture and surfacing can race. The files are not locked and updates are not atomic.

6.6 Documented features not yet implemented

The architecture describes deduplication, recurrence, provenance processing, ranking, and counterdreams. The executable currently:

- does not detect recurrence;
- does not deduplicate;
- does not use `trust_level`;
- does not sort survivors by score;
- writes `counterdream: null`;
- does not archive non-survivors;
- does not emit a durable run manifest.

This is acceptable for a plainly labelled scaffold, but unattended cron activation should wait until the executable path satisfies the minimum lifecycle guarantees.

6.7 Seed data and clean installation

The alpha fragments are valuable project history. They also include project-specific assumptions and personally framed material. A new installation should not inherit them as active runtime memory.

Recommended layout:

```
templates/  
  DREAMS.md  
  SURFACE.md  
  DREAMS_DAY.md  
  SUBCONSCIOUS.md  
  DREAMS_ARCHIVE.md  
examples/  
  alpha-session/  
    DREAMS.seed.md
```

```
runtime/
# generated locally and ignored by git
```

7 v0.2 Reference Architecture

7.1 Package structure

```
hum/
src/hum/
  schema.py      # typed fragment and lifecycle models
  parser.py      # Markdown/YAML parser with source spans
  store.py       # locks, journal, atomic persistence
  scoring.py     # evidence-derived scoring
  recurrence.py  # exact and optional semantic matching
  lifecycle.py   # validated state transitions
  security.py    # origin and authority enforcement
  reports.py     # morning summaries and run manifests
scripts/
  hum-capture
  hum-surface
  hum-validate
  hum-promote
templates/
examples/
tests/
```

The exact language and packaging can remain lightweight. The important change is to move lifecycle-critical file manipulation out of model-authored instructions and into deterministic code.

7.2 State machine

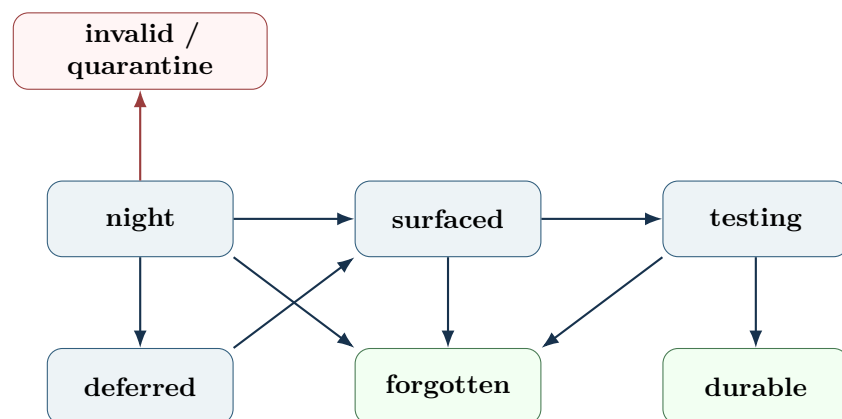


Figure 3: Recommended explicit lifecycle states. Every input fragment receives a recoverable disposition.

Transitions should be validated. For example, a fragment in the night state may become surfaced, deferred, forgotten, or invalid; it should not jump directly to durable memory.

7.3 Executable capture

A `hum-capture` command should own:

- collision-resistant ID generation;
- local timestamp and offset;
- accepted type validation;
- source and trust normalization;
- body hashing;
- file locking;
- atomic append;
- duplicate detection;
- final parse verification.

A time-based ID with a random suffix avoids shared sequential counters:

`dream-20260716T221400-7K3F`

7.4 Transactional surfacing

The morning pass should follow a write-ahead workflow:

1. acquire an exclusive store lock;
2. assign a unique `run_id`;
3. read and hash all inputs;
4. parse and validate records;
5. classify invalid records into quarantine;
6. calculate evidence-derived signals;
7. deduplicate and group recurrence;
8. rank and allocate the daily budget;
9. prepare complete output files in a temporary transaction directory;
10. verify that every input ID has exactly one disposition;
11. `fsync` and atomically replace outputs;

12. write a run manifest containing input and output hashes;
13. release the lock.

Listing 2: Conceptual surfacing transaction.

```
def surface_cycle(store, policy):
    with store.exclusive_lock():
        run = store.begin_transaction()
        records = store.read_night_fragments()
        parsed, invalid = validate_all(records)

        groups = detect_recurrence(parsed)
        scored = score_from_evidence(groups, policy)
        surfaced, deferred, forgotten = allocate(scored, policy)

        run.stage("SURFACE.md", surfaced)
        run.stage("DREAMS.md", deferred)
        run.stage("DREAMS_ARCHIVE.md", tombstones(forgotten))
        run.stage("DREAMS_QUARANTINE.md", invalid)
        run.assert_total_disposition(records)
        run.commit_atomic()
```

7.5 Run manifest and idempotency

Each run should create a small immutable manifest:

```
run_id: surface-20260717T070000-2M8Q
started_at: 2026-07-17T07:00:00+02:00
input_hash: sha256:...
policy_version: surface-v0.2.0
counts:
  total: 18
  surfaced: 4
  deferred: 11
  forgotten: 2
  invalid: 1
output_hashes:
  SURFACE.md: sha256:...
  DREAMS.md: sha256:...
```

If the same input hash and policy version are seen again, the process can detect a retry rather than duplicating work.

8 Selection and Scoring

8.1 Why a score is not enough

A scalar score is useful for ordering candidates. It is not a complete decision policy. Different fragment classes have different semantics: a high-consequence warning may require review despite low novelty, while five near-identical associations should not consume the entire morning budget.

The v0.2 policy should combine:

1. hard eligibility gates;
2. evidence-derived component scores;
3. type-specific rules;
4. diversity constraints;
5. a daily review budget;
6. a small control sample from below the cutoff.

8.2 Eligibility gates

Before scoring, require:

- parseable and schema-valid metadata;
- unique ID and body hash;
- accepted type and state;
- non-empty concise body;
- known source class;
- no prohibited secret or credential pattern;
- no attempt to grant the fragment authority;
- no unresolved corruption or duplicate conflict.

8.3 Evidence-derived score

A revised score can retain the protocol’s conceptual variables while avoiding positive assumptions:

$$S = \sigma (\beta_0 + \beta_r R + \beta_n N + \beta_u U + \beta_a A + \beta_c C + \beta_p P + \beta_v V - \beta_d D - \beta_x X - \beta_q Q), \quad (4)$$

where σ is a logistic bounding function. Components should be null until evidence exists. The system may impute zero for ranking, but the record must distinguish **unknown** from observed zero.

Term	Meaning	Evidence examples
<i>R</i>	Recurrence	Exact or semantic match across independent tasks; capped logarithmically.
<i>N</i>	Novelty	Low similarity to existing durable memory and current active notes.
<i>U</i>	Expected utility	A concrete future decision, test, or workflow could use the fragment.
<i>A</i>	Active relevance	Match to named project, current objective, or unresolved task.
<i>C</i>	Consequence	Risk severity, cost of recurrence, or magnitude of opportunity.
<i>P</i>	Predictive / user signal	Later event predicted, explicit user endorsement, or confirmed correction.
<i>V</i>	Validation quality	Independent evidence, reproducible outcome, or successful waking test.
<i>D</i>	Decay	Time since last relevance, adjusted by type and recurrence.
<i>X</i>	Contradiction	Counterevidence, failed tests, or scope conflict.
<i>Q</i>	Provenance risk	Untrusted origin, unclear derivation, or possible injection.

8.4 Daily budget and diversity

Rather than surface every item above a fixed threshold, HUM should select a bounded set such as:

- top three general candidates;
- up to one mandatory high-consequence warning;
- up to one returning behavioural pattern;
- one randomly sampled below-cutoff control fragment on selected days.

A maximal-marginal-relevance rule or simple type/project quotas can prevent one recurring theme from monopolizing attention.

8.5 Counterdreams

A counterdream is a plausible alternative explanation or disconfirming test. It should be generated only for selected high-weight items, not for every captured fragment. The counterdream should be stored as a hypothesis and reviewed independently.

Examples:

- The apparent behavioural default may be caused by underspecified prompts.
- Repetition may result from copied context rather than independent recurrence.
- A project-specific preference may not generalize to other tasks.
- A warning may already be represented in `SOUL.md` and therefore add no novelty.

9 Promotion, Retrieval, and Controlled Forgetting

9.1 Promotion to durable memory

A fragment should reach `SUBCONSCIOUS.md` only after at least one waking test and adequate provenance. High-impact behavioural rules should require two independent signals, for example recurrence plus a demonstrated decision improvement.

Promotion metadata should include:

- originating fragment IDs;
- evidence for and against;
- tests and outcomes;
- confidence and valence;
- scope and trigger conditions;
- retrieval exclusions;
- expiry or review date;
- authority class.

9.2 Preference scope

A user correction is not automatically a global preference. Preference memory should distinguish:

- current-turn instruction;
- task-local preference;
- named project preference;
- domain preference;
- global durable preference explicitly confirmed by the user.

Scope errors are among the most likely ways a useful memory system becomes irritating or unsafe.

9.3 Retrieval policy

Retrieval should consider:

$$\text{retrieve}(m, t) = f(\text{semantic match}, \text{scope}, \text{recency}, \text{confidence}, \text{authority}, \text{conflict}, \text{expiry}). \quad (5)$$

A retrieved item should be accompanied by a compact citation to its own provenance and last validation. The agent should be able to distinguish “this was explicitly requested by the user” from “this was inferred from repeated behaviour.”

9.4 Forgetting and tombstones

A full fragment can be removed when it is low-value, obsolete, duplicated, or contradicted. A tombstone should retain only what is needed to avoid wasteful rediscovery:

```
id: dream-20260710T143000-AB19
dismissed_at: 2026-07-13T21:10:00+02:00
reason: contradicted_by_repository_inspection
content_hash: sha256:...
source_class: internal_observation
```

Tombstones should also expire when they no longer serve a deduplication or audit purpose.

10 Evaluation Framework

10.1 Evaluation objective

HUM should be judged by whether it improves later behaviour without imposing excessive noise, rigidity, or security risk. Interesting morning summaries are not sufficient evidence of value.

Recent benchmarks increasingly distinguish passive recall from active memory use. LoCoMo evaluates long-term conversation understanding [6]; Mem2ActBench emphasizes proactive memory application in tool-based tasks [8]; Agentic Memory treats memory operations as explicit agent actions [9]; and LoCoMo-Plus examines latent constraints that must be applied despite semantic disconnect between cue and later trigger [10]. HUM’s evaluation should reflect this shift from recall to action quality.

10.2 Primary metrics

Metric	Definition
Surfacing precision	Fraction of surfaced fragments later judged useful, predictive, corrective, or worthy of continued testing.
Control recovery rate	Fraction of sampled below-cutoff fragments later found useful; estimates false omission.
Decision impact rate	Fraction of surfaced fragments that measurably alter or improve a later decision.
Repeat-error reduction	Change in recurrence rate of known behavioural failures after corrective heuristics are activated.
Noise burden	Human or agent review time spent on fragments ultimately dismissed without value.
Memory conflict rate	Frequency with which retrieved durable memories conflict with current instructions, newer preferences, or each other.
Scope error rate	Frequency with which local preferences are incorrectly applied globally.
Poisoning acceptance rate	Fraction of adversarial or untrusted memory writes that enter durable storage or influence later actions.
Retrieval-to-action utility	Success rate when memory must be proactively applied to tool choice or parameter grounding.
Storage efficiency	Durable bytes and retrieval tokens per validated useful memory.
Audit completeness	Fraction of durable memories with complete origin, lineage, test, scope, and review metadata.

10.3 Four-phase experiment

10.3.1 Phase 1: manual baseline

Duration: approximately two weeks.

- Use deterministic capture but manual surfacing and disposition.
- Record why each fragment was surfaced or dismissed.
- Establish natural fragment volume, recurrence, and review burden.
- Do not promote automatically.

10.3.2 Phase 2: shadow scoring

- Run the automatic scorer without changing files.
- Compare ranked output with human selections.
- Tune weights and budgets using held-out days.
- Include random control fragments to estimate false negatives.

10.3.3 Phase 3: bounded automatic surfacing

- Enable atomic morning transactions.
- Limit surfaced items to a small daily budget.
- Require human review of invalid, warning, and high-provenance-risk records.
- Keep promotion manual.

10.3.4 Phase 4: controlled promotion trial

- Permit automatic promotion only for low-consequence, tightly scoped patterns that satisfy explicit tests.
- Require approval for global preferences, security-relevant rules, and `SOUL.md` proposals.
- Measure retrieval impact and regressions over several weeks.

10.4 Ablation studies

Useful ablations include:

- recurrence on versus off;
- counterdreams on versus off;
- fixed threshold versus daily budget;
- diversity constraint versus pure score ranking;
- tombstones versus complete deletion;
- provenance penalty versus no provenance penalty;
- human promotion versus automatic promotion.

11 Implementation Roadmap

11.1 v0.2: safe lifecycle mechanics

1. strict schema and state validation;
2. executable capture command;
3. quarantine file and error reporting;
4. locking, temporary files, `fsync`, and atomic replacement;
5. run manifests and idempotent retries;
6. explicit surfaced, deferred, forgotten, and invalid dispositions;

7. corrected evidence-derived scoring;
8. daily surfacing budget and sorted output;
9. unit and fault-injection tests;
10. alpha seed material moved to examples.

11.2 v0.3: recurrence and reflective evaluation

1. exact recurrence using normalized hashes;
2. optional semantic recurrence with transparent similarity thresholds;
3. counterdream generation for selected candidates;
4. day-stage test and outcome workflow;
5. scope-aware preference modelling;
6. retrieval citations and conflict detection.

11.3 v0.4: secure durable memory

1. origin-bound authority labels across transformations;
2. memory-poisoning test suite;
3. retrieval-time authority filtering;
4. automated expiry and review scheduling;
5. controlled low-consequence promotion;
6. dashboards for utility, error reduction, noise, and risk.

11.4 v1.0: validated slow-cognition subsystem

A v1.0 designation should require evidence that HUM:

- improves at least one meaningful behavioural metric;
- does not increase memory conflict or scope errors beyond an agreed bound;
- survives interrupted writes and retries without loss or duplication;
- resists defined memory-poisoning attacks;
- supports complete audit lineage for durable memories;
- permits clean export, deletion, and reconstruction;
- has a stable governance procedure for normative changes.

12 Open Research Questions

12.1 Can weak associations be evaluated without suppressing creativity?

Aggressive evidence requirements favour behavioural corrections and concrete warnings over speculative associations. That is appropriate for durable memory but may suppress the most creative part of the system. Cross-dreams may need a separate low-authority incubation pool with longer expiry and evaluation through later usefulness rather than immediate verifiability.

12.2 How should recurrence be defined?

Exact text recurrence is too narrow. Embedding similarity can conflate distinct ideas or be manipulated by repeated paraphrase. A robust system may combine normalized hashes, entity and relation overlap, task independence, and human-confirmed cluster labels.

12.3 What is the optimal review budget?

A fixed number of morning fragments may be too rigid when workload varies. A dynamic budget could depend on active project count, unresolved warnings, previous review backlog, and the measured yield of recent surfacing.

12.4 When does a behavioural pattern become a rule?

Repeated corrections may reflect a stable agent defect, an underspecified user request, or a temporary project condition. HUM needs cross-context tests before generalization. The system should prefer narrow triggers over broad personality changes.

12.5 Can the system detect lucidity debt?

A particularly valuable metric may be the number of times the agent recognizes a known pattern but fails to apply the corrective heuristic. This “lucidity debt” separates memory retrieval failure from action-policy failure.

12.6 How much biological metaphor is useful?

The dream language makes the architecture memorable and encourages deferred reflection. It can also invite unjustified anthropomorphic explanations. The design should preserve poetic names at the interface while maintaining operational definitions in code, logs, and evaluation.

13 Limitations

HUM does not solve truth verification. A carefully governed false memory remains false. It does not replace external evidence, user confirmation, or task-specific validation. It also does not guarantee creativity; incubation mechanisms can generate attractive but useless associations.

The Markdown source-of-truth design is appropriate for a personal agent but will eventually face performance and concurrency limits. A derived index will likely become useful for semantic recurrence and retrieval at scale. Such an index should remain reconstructible and must not silently diverge from canonical state.

The current evaluation plan depends partly on human judgement of usefulness. This introduces subjectivity and possible hindsight bias. Control fragments, blind ratings, explicit predicted outcomes, and predeclared metrics can reduce but not eliminate that problem.

Finally, research on memory poisoning is rapidly evolving. Several relevant 2026 results are recent preprints or newly published conference work. HUM's security model should be tested continuously rather than treated as complete.

14 Conclusion

HUM offers a coherent answer to a problem that persistent agents increasingly face: useful experience accumulates faster than it can safely become memory. The DREAMS protocol creates an intermediate domain in which unfinished observations can exist without being believed, repeated patterns can become visible without becoming compulsive, and potential rules can be tested before they influence future behaviour.

The architecture's enduring value is not the dream metaphor by itself. It is the separation of capture, attention, evaluation, persistence, and authority. That separation makes slow cognition governable.

The repository scaffold is an excellent starting point. Its protocol and capture boundary are already stronger than many memory systems that treat reflection text as inherently trustworthy. The next engineering step should be deliberately unromantic: strict schemas, atomic transactions, recoverable disposition, origin-bound authority, and measured selection. Once those foundations are proven, the richer mechanisms - counterdreams, cross-domain incubation, lucid pattern recognition, and controlled promotion - can be added without turning the agent's memory into an unauditable narrative engine.

Final principle

Preserve the poetry, but harden the epistemology. Capture without believing. Surface without accepting. Remember without granting authority. Forget without losing accountability.

A Normative Requirements Checklist

ID	Requirement	Priority
R-01	Every fragment has a unique ID, valid type, timestamp with offset, source class, trust level, authority, and body hash.	Must
R-02	Invalid records are quarantined without modification or loss.	Must
R-03	Every surfacing input receives exactly one disposition.	Must
R-04	File updates are locked, journaled, verified, and atomically committed.	Must
R-05	Re-running the same transaction cannot duplicate or lose records.	Must
R-06	Raw fragments and external content cannot override instructions or policy.	Must
R-07	Origin and authority survive summarization, recurrence grouping, and promotion.	Must
R-08	Scores use observed evidence and distinguish unknown from zero.	Must
R-09	Surfacing is bounded by a review budget and diversity policy.	Should
R-10	Durable memory includes scope, triggers, evidence, contradiction, confidence, expiry, and lineage.	Must
R-11	SOUL.md changes are never automatic.	Must
R-12	The system supports controlled deletion and auditable tombstones.	Must
R-13	A control-sampling mechanism estimates false omissions.	Should
R-14	Security evaluation includes direct, sleeper, and laundering memory-poisoning scenarios.	Must
R-15	The canonical store can be reconstructed independently of any derived index.	Must

B Suggested v0.2 Policy Configuration

```

policy_version: surface-v0.2.0
surfacing:
  daily_budget: 4
  warning_reserve: 1
  behavioural_reserve: 1
  control_sample_probability: 0.20
  max_per_project: 2
  max_per_type: 2
eligibility:
  require_source: true
  require_authority: true
  accepted_states: [night, deferred]
  accepted_types:
    - OBSERVATION

```

```

- BEHAVIOUR
- ASSOCIATION
- QUESTION
- CONTRADICTION
- RESISTANCE
- SEED
- WARNING
security:
  quarantine_unknown_origin: true
  external_default_authority: descriptive_only
  allow_memory_to_authorize_actions: false
promotion:
  automatic: false
  minimum_independent_signals: 2
  require_counterdream: true
  require_review_after: true

```

C Minimum Test Matrix

Test	Expected behaviour
Malformed YAML	Original record copied to quarantine with parse error; no data loss.
Unknown fragment type	Schema rejection and quarantine.
Duplicate ID, same body	Deduplicated or linked as exact recurrence according to policy.
Duplicate ID, different body	Hard conflict; transaction stops or quarantines both records.
Body contains delimiter line	Parser preserves body and does not split incorrectly.
Concurrent capture during surfacing	Lock or append journal prevents race and lost write.
Crash before commit	Canonical files remain unchanged.
Crash during commit	Recovery uses transaction journal and hashes.
Retry same run	Idempotent no-op or verified completion.
Missing output file	Created from template without discarding input.
Untrusted prompt injection	Stored, if at all, as untrusted descriptive content; cannot gain authority.
Laundered summary	Derivation retains original external origin.
Manufactured repetition	Recurrence capped; independent-source requirement prevents automatic trust increase.
Expired preference	Excluded from retrieval pending review.

Test	Expected behaviour
Conflicting durable memories	Retrieval exposes conflict and defers to current instruction.

D Glossary

Authority

The degree to which an item is permitted to influence or authorize behaviour; separate from factual confidence.

Counterdream

A plausible alternative explanation, contradiction, or disconfirming test attached to a surfaced fragment.

Dream fragment

A compact low-authority observation, association, question, warning, or behavioural trace retained for later evaluation.

Durable memory

A validated, scoped, provenance-bearing item in SUBCONSCIOUS.md; not a normative rule by default.

Lucidity debt

A recognized behavioural pattern for which the corrective heuristic was not applied.

Origin binding

Preservation of the original source and trust class through summaries, transformations, and retrieval.

Surfacing

Selection of a fragment for conscious review; not acceptance as true.

Tombstone

Minimal record that an item was dismissed, superseded, or deleted, retaining only audit and deduplication fields.

Valence

Operational importance or consequence, not an assertion of human-like emotion.

Waking test

A later observation or experiment that evaluates whether a fragment predicts, improves, prevents, or explains something useful.

References

- [1] S. Diekelmann and J. Born. The memory function of sleep. *Nature Reviews Neuroscience*, 11:114–126, 2010. doi: [10.1038/nrn2762](https://doi.org/10.1038/nrn2762).
- [2] J. G. Klinzing, N. Niethard, and J. Born. Mechanisms of systems memory consolidation during sleep. *Nature Neuroscience*, 22:1598–1610, 2019. doi: [10.1038/s41593-019-0467-3](https://doi.org/10.1038/s41593-019-0467-3).
- [3] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of UIST 2023*; arXiv:2304.03442, 2023.
- [4] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36*, 2023; arXiv:2303.11366.
- [5] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
- [6] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of ACL 2024*, pages 13851–13870, 2024. doi: [10.18653/v1/2024.acl-long.747](https://doi.org/10.18653/v1/2024.acl-long.747).
- [7] Z. Jia, Q. Liu, H. Li, Y. Chen, and J. Liu. Evaluating the Long-Term Memory of Large Language Models. In *Findings of ACL 2025*, 2025.
- [8] Y. Shen, K. Li, W. Zhou, and S. Hu. Mem2ActBench: A Benchmark for Evaluating Long-Term Memory Utilization in Task-Oriented Autonomous Agents. In *Proceedings of ACL 2026*, pages 8173–8190, 2026. doi: [10.18653/v1/2026.acl-long.370](https://doi.org/10.18653/v1/2026.acl-long.370).
- [9] Y. Yu, L. Yao, Y. Xie, Q. Tan, J. Feng, Y. Li, and L. Wu. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. In *Proceedings of ACL 2026*, pages 21457–21483, 2026. doi: [10.18653/v1/2026.acl-long.981](https://doi.org/10.18653/v1/2026.acl-long.981).
- [10] Y. Li, W. Guo, L. Zhang, R. Xu, M. Huang, H. Liu, L. Xu, Y. Xu, and J. Liu. LoCoMo-Plus: Beyond-Factual Cognitive Memory Evaluation Framework for LLM Agents. In *Proceedings of ACL 2026*, pages 25085–25100, 2026. doi: [10.18653/v1/2026.acl-long.1150](https://doi.org/10.18653/v1/2026.acl-long.1150).
- [11] B. D. Sunil, I. Sinha, P. Maheshwari, S. Todmal, S. Malik, and S. Mishra. Memory Poisoning Attack and Defense on Memory Based LLM-Agents. arXiv:2601.05504, 2026.
- [12] P. Dash, T. Ge, A. Jain, T. Shah, and Z. Shang. From Untrusted Input to Trusted Memory: A Systematic Study of Memory Poisoning Attacks in LLM Agents. arXiv:2606.04329, 2026.
- [13] S. Pulipaka, S. Hlebik, L. Raghav, S. Abdelnabi, V. Raina, I. Sheth, and M. Fritz. Hidden in Memory: Sleeper Memory Poisoning in LLM Agents. arXiv:2605.15338, 2026.

- [14] C. Stone. HUM: DREAMS agent subsystem, initial scaffold. GitHub repository, commit 3bfce73ae61b6da03b7e65523ca5f690783c71b0, July 2026. <https://github.com/nobulart/hum>.